



MISO

Maestría en Ingeniería de Software

PROYECTO - ENTREGA 5

Plan de pruebas, Análisis de capacidad y
Resultados

GRUPO 11

SERGIO FERNANDO BARRERA MOLANO
HAROLD ANDRÉS BARTOLO MOSCOSO
EDWIN HERNÁN HURTADO CRUZ
JUAN JOSÉ RESTREPO BONILLA

202517034
202513889
202326341
202516633

Plan de pruebas, Análisis de capacidad y Resultados

1. Objetivo general de las pruebas.

Diseñar, ejecutar y documentar escenarios de prueba que permitan determinar la capacidad máxima soportada por el sistema desplegado sobre AWS ECS + Fargate.

Para ello se diseñaron las siguientes pruebas:

- Acceso simultáneo
- Carga progresiva
- Estrés extremo
- Utilización de recursos críticos (CPU, RAM, red, cola SQS, base de datos RDS).

Lo anterior, para poder identificar cuellos de botella y límites del sistema, determinando:

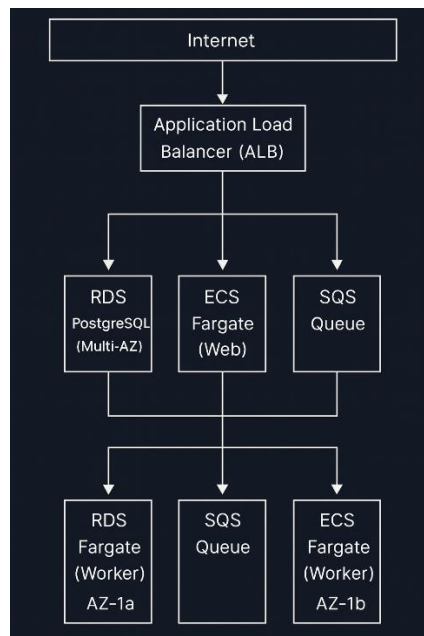
- Número máximo de usuarios concurrentes
- Número máximo de videos procesados por unidad de tiempo
- Métricas claves antes del punto de degradación.

Finalmente realizar la documentación de los resultados mediante:

- Gráficas
- Análisis comparativo
- Conclusiones técnicas

2. Arquitectura Bajo Prueba (PaaS AWS Fargate).

Esta sección describe la infraestructura y las condiciones bajo las cuales se realizaron las pruebas, igualmente la arquitectura se encuentra en el siguiente [link](#).



Componente	Descripción
Región AWS	us-east-1
Balanceador de carga (ALB)	Distribuye peticiones HTTP hacia las instancias EC2 de la capa web
Instancias EC2 (Auto Scaling Group)	3 instancias t3.small para la capa web
Base de datos (RDS)	PostgreSQL (db.t3.small) utilizada para autenticación y almacenamiento persistente
Almacenamiento de objetos (S3)	Contiene los archivos asociados a los usuarios
Monitoreo (CloudWatch)	Recolección de métricas de CPU, red y respuesta del balanceador
Amazon SQS o Amazon Kinesis	Intercambio de mensajes de forma confiable, rápida y asíncrona.
Herramienta de prueba	Apache JMeter 5.6.3 ejecutado en Windows
Tipo de prueba	EC1: Estrés (simulación progresiva de múltiples usuarios intentando autenticarse simultáneamente) EC2: Estrés (Aumento de carga de videos a procesar / min)

La arquitectura actual incluye:

- Capa Web: ECS Fargate – API (0.5 vCPU / 1GB RAM)
- Capa Worker: ECS Fargate – Procesamiento de video (1 vCPU / 2GB RAM)
- Mensajería: Amazon SQS
- Base de Datos: RDS PostgreSQL (Multi-AZ)
- Storage: Amazon S3

El análisis se centra en carga web y procesamiento de video en workers.

3. Herramientas de pruebas usadas (JMETER).

Para la realización de las pruebas se usó el software JMETER, instalado de forma local en los equipos, de modo que se lanza la ejecución sobre el AWS. En especial Apache JMeter 5.6.3 ejecutado en Windows 11

4. Entorno de Pruebas.

Esta sección describe la infraestructura y las condiciones bajo las cuales se realizaron las pruebas en los 02 entornos.

4.1. Escenario 1 – Carga Web / API

Objetivo: identificar la cantidad máxima de usuarios simultáneos que pueden invocar la API sin degradar rendimiento, en este escenario se probaron los siguientes Endpoints:

- POST /videos (subida de video)
- GET /videos/{id}
- GET /health (baseline)

Métrica principal: Tiempo de respuesta (ms) y porcentaje de errores / Obtenido mediante JMETER.

Carga: Ramp-up: 1, 50, 100, 200, etc. Usuarios simultáneos

Límite de la prueba: Hasta observar

- 1s respuesta promedio
- 5% errores (O superior / Por lo general es una exponencial de errores)
- Saturación CPU > 80% en Fargate (web)

4.2. Escenario 2 – Procesamiento de Video (Workers)

Objetivo: Medir cuántos videos por minuto procesan los workers antes de saturarse, para ello se utilizó un video de más de 20 segundos y con formato distinto a 16:9 (el que ya teníamos prediseñado en pruebas anteriores).

La prueba de este escenario está compuesta por:

1. Insertar videos de forma masiva (desde la API).
2. Observar SQS:
 - Mensajes enviados
 - Mensajes en cola
 - Tiempo de procesamiento
3. Escalar carga:
 - 01 video/min, 05 videos/min, 10 videos/min, 20 videos/min, 40 videos/min, etc
4. Métricas:
 - Videos/min procesados
 - Tiempo promedio de procesamiento (Bajo porcentaje de carga)
 - Profundidad máxima de la cola (Antes de saturarse)
 - CPU Worker > 80% (Con # número de videos en procesamiento + cola)

5. Ejecución del escenario de prueba 01.

En esta sección se describe el diseño del escenario 01 que se utilizó para someter la aplicación a condiciones de carga alta / extrema.

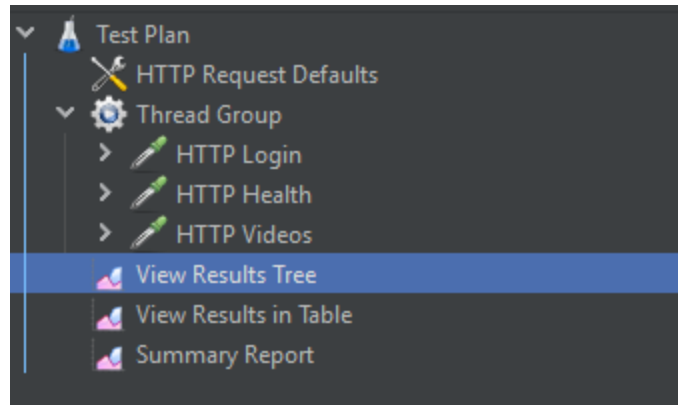
Se tuvieron 02 enfoques así:

1. Realizar peticiones de Login (estrés a capa Web de Login), posterior con el bearer obtenido realizar la petición de estatus de la aplicación (estrés capa API) y finalizar realizar la petición de los videos de la aplicación (estrés del API) para estresar la parte de consultas hacia el backend de la aplicación.
2. Realizar el Login de forma manual con el FastAPI para obtención del bearer, y con este realizar la petición de estatus de la aplicación (estrés capa API) y finalizar realizar la petición de los videos de la aplicación (estrés del API) para estresar la parte de consultas hacia el backend de la aplicación.

Estos enfoques se realizaron dado que se evidencio previamente en la entrega pasada que el API de logeo y generación del token de autenticación por lo generar tiende a fallar antes que los API de la capa Web diseñados (funcionalidad del APP).

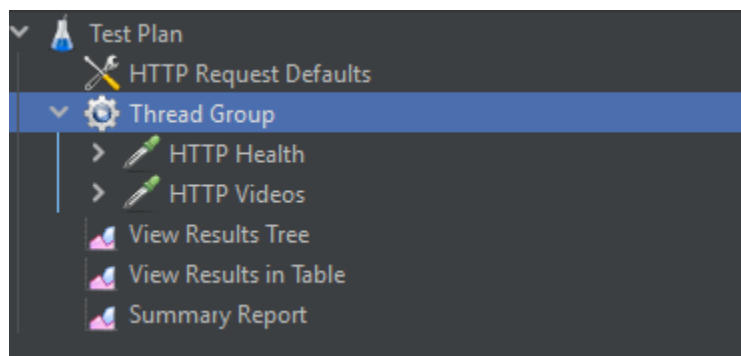
En la siguiente imagen se puede apreciar la configuración de Jmeter, donde se aprecia el Thread Group, el cual ejecuta las pruebas.

Enfoque 01: Estrés al API de autorización y generación de token.



En esta configuración cada “thread” o usuario realiza el Login para obtener el bearer y con ello realizar el consumo de las API de Health y Videos.

Enfoque 02: Estrés al API de /health y obtención de /videos



En esta configuración cada “thread” o usuario necesita a priori el bearer de autorización, el cual se genera de forma manual, con ellos realizar el consumo de las API de Health y Videos.

6. Resultados - Análisis y Cuellos de Botella.

A continuación, se presentan los resultados obtenidos durante la ejecución de la prueba. Se incluyen las métricas de rendimiento, los errores detectados y el comportamiento general de la aplicación ante el aumento de carga.

Al iniciar el proceso de sobrecarga de forma gradual, observamos que el aplicativo responde correctamente, sin embargo, al intentar realizar un aumento repentino (Spike) de 1000 usuarios en un tiempo rampa de 20 segundos (50 usuarios ingresando por segundo) al llegar a más de 112 usuarios concurrentes se degrada la aplicación, como se observa en las siguientes imágenes, evidenciamos que el elemento que falla inicialmente es el autorizador (Login para generar bearer).

Sample #	Start Time	Thread Name	Label	Sample Time(s)	Status	Bytes	Sent Bytes	Latency	Connect Time(s)
887	23/12/24 22:21	Thread Group 1-412	HTTP Health	205	Success	196	247	205	83
888	23/12/24 22:22	Thread Group 1-421	HTTP Login	60206	Error	292	294	60206	82
889	23/12/24 22:20	Thread Group 1-413	HTTP Health	196	Success	196	247	196	86
890	23/12/24 22:24	Thread Group 1-422	HTTP Login	60184	Error	292	294	60184	83
891	23/12/24 22:22	Thread Group 1-423	HTTP Login	60186	Error	292	294	60186	83
892	23/12/24 22:24	Thread Group 1-424	HTTP Login	60181	Error	292	294	60181	81
893	23/12/24 22:22	Thread Group 1-416	HTTP Health	188	Success	196	247	188	84
894	23/12/24 22:22	Thread Group 1-415	HTTP Health	196	Success	196	247	196	84
895	23/12/24 22:22	Thread Group 1-414	HTTP Health	196	Success	196	247	196	84
896	23/12/24 22:24	Thread Group 1-425	HTTP Login	60183	Error	292	294	60183	82
897	23/12/24 22:24	Thread Group 1-426	HTTP Login	60184	Error	292	294	60184	81
898	23/12/24 22:24	Thread Group 1-427	HTTP Login	60184	Error	292	294	60184	81
899	23/12/24 22:24	Thread Group 1-428	HTTP Health	195	Success	196	247	195	80
900	23/12/24 22:22	Thread Group 1-417	HTTP Health	201	Success	196	247	201	116
901	23/12/24 22:28	Thread Group 1-419	HTTP Health	198	Success	196	247	198	83
902	23/12/24 22:24	Thread Group 1-429	HTTP Health	205	Success	196	247	205	82
903	23/12/24 22:21	Thread Group 1-428	HTTP Login	60208	Error	292	294	60208	81
904	23/12/24 22:22	Thread Group 1-429	HTTP Login	60180	Error	292	294	60180	82
905	23/12/24 22:24	Thread Group 1-429	HTTP Login	60187	Error	292	294	60187	82
906	23/12/24 22:24	Thread Group 1-421	HTTP Login	60186	Error	292	294	60186	83
907	23/12/24 22:28	Thread Group 1-421	HTTP Health	198	Success	196	247	198	84
908	23/12/24 22:24	Thread Group 1-422	HTTP Health	188	Success	196	247	188	84
909	23/12/24 22:24	Thread Group 1-422	HTTP Health	198	Success	196	247	198	82
910	23/12/24 22:24	Thread Group 1-423	HTTP Login	60184	Error	292	294	60184	82
911	23/12/24 22:24	Thread Group 1-423	HTTP Login	60183	Error	292	294	60183	81
912	23/12/24 22:24	Thread Group 1-424	HTTP Login	60186	Error	292	294	60186	84
913	23/12/24 22:24	Thread Group 1-424	HTTP Health	212	Success	196	247	212	123
914	23/12/24 22:22	Thread Group 1-421	HTTP Health	192	Success	196	247	191	84
915	23/12/24 22:24	Thread Group 1-426	HTTP Health	172	Success	196	247	172	84
916	23/12/24 22:22	Thread Group 1-425	HTTP Login	60211	Error	292	294	60211	83
917	23/12/24 22:24	Thread Group 1-427	HTTP Health	205	Success	196	247	205	82
918	23/12/24 22:24	Thread Group 1-428	HTTP Login	60191	Error	292	294	60191	82
919	23/12/24 22:22	Thread Group 1-427	HTTP Login	60170	Error	292	294	60170	84
920	23/12/24 22:21	Thread Group 1-428	HTTP Login	60184	Error	292	294	60184	83
921	23/12/24 22:21	Thread Group 1-429	HTTP Health	198	Success	196	247	198	84

JMeter en punto de quiebre, estrés medio alto

Test	Sampler result	Request	Response data
HTTP Health	205	196	247
HTTP Login	60206	292	294
HTTP Health	196	196	247
HTTP Login	60184	292	294
HTTP Login	60186	292	294
HTTP Login	60181	292	294
HTTP Health	188	196	247
HTTP Health	196	196	247
HTTP Health	196	196	247
HTTP Login	60183	292	294
HTTP Login	60184	292	294
HTTP Health	195	196	247
HTTP Health	201	196	247
HTTP Health	198	196	247
HTTP Health	205	196	247
HTTP Login	60208	292	294
HTTP Login	60180	292	294
HTTP Login	60187	292	294
HTTP Login	60186	292	294
HTTP Health	198	196	247
HTTP Health	188	196	247
HTTP Health	198	196	247
HTTP Login	60184	292	294
HTTP Login	60183	292	294
HTTP Login	60186	292	294
HTTP Health	212	196	247
HTTP Health	192	196	247
HTTP Health	172	196	247
HTTP Login	60211	292	294
HTTP Health	205	196	247
HTTP Login	60191	292	294
HTTP Login	60170	292	294
HTTP Login	60184	292	294
HTTP Health	198	196	247

JMeter cuando la aplicación está completamente caída

A continuación, presento la vista desde la maquina donde se evidencia la ejecución de las pruebas de estrés al API Web:


```
root@ip-10-0-143-111:/home/ubuntu# systemctl status anb-worker-sqs
● anb-worker-sqs.service - ANB Sqs worker service
   Loaded: loaded (/etc/systemd/system/anb-worker-sqs.service; enabled; preset: enabled)
   Active: active (running) since Sun 2025-11-16 05:18:56 UTC; 3min 48s ago
     Main PID: 12189 (start-worker.sh)
        Tasks: 2 (limit: 2204)
      Memory: 764.0k (peak: 11.4M)
         CPU: 524ms
    CGroup: /system.slice/anb-worker-sqs.service
            └─12189 /bin/bash /opt/anb-worker/start-worker.sh
              └─17392 sleep 10

Nov 16 05:22:42 ip-10-0-143-111 start-worker.sh[12236]: --> 6092174c71cb
Nov 16 05:22:42 ip-10-0-143-111 start-worker.sh[12236]: Step 19/19 : CMD ["/app/scripts/start.sh"]
Nov 16 05:22:42 ip-10-0-143-111 start-worker.sh[12236]: --> Running in ec0311304333
Nov 16 05:22:42 ip-10-0-143-111 start-worker.sh[12236]: --> Removed intermediate container ec0311304333
Nov 16 05:22:42 ip-10-0-143-111 start-worker.sh[12236]: --> 88d4d226e918
Nov 16 05:22:42 ip-10-0-143-111 start-worker.sh[12236]: Successfully built 88d4d226e918
Nov 16 05:22:42 ip-10-0-143-111 start-worker.sh[12236]: Successfully tagged anb-worker:latest
Nov 16 05:22:42 ip-10-0-143-111 start-worker.sh[12189]: Starting Sqs worker container...
Nov 16 05:22:42 ip-10-0-143-111 start-worker.sh[12236]: dc3489bfff79d7a9d11dc4b029e89dec0cb9793c55e6499b74e53fd75d9727db
Nov 16 05:22:43 ip-10-0-143-111 start-worker.sh[12189]: Sqs worker service started successfully!
root@ip-10-0-143-111:/home/ubuntu# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
dc3489bfff79d  anb-worker:latest  python -m app.worke...  6 seconds ago  Up 5 seconds  8000/tcp       anb-worker-sqs
root@ip-10-0-143-111:/home/ubuntu# docker logs -f anb-worker-sqs
2025-11-16 05:22:44,738 - _main_ - INFO - Starting Sqs worker...
2025-11-16 05:22:44,738 - _main_ - INFO - Sqs Queue URL: https://sqs.us-east-1.amazonaws.com/906569879596/anb-video-processing-queue
2025-11-16 05:22:44,738 - _main_ - INFO - Sqs Region: us-east-1
```

Consola de la aplicación, cuando se arranca.

```
INFO: 10.0.28.35:60930 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:60994 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:60990 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:60974 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:61014 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:61004 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:60968 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:60956 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:61040 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:61072 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:61056 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:61026 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: 10.0.28.35:61078 - "GET /api/videos HTTP/1.1" 401 Unauthorized
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: 10.0.28.35:39334 - "GET /api/videos HTTP/1.1" 200 OK
INFO: 10.0.28.35:62446 - "GET /api/videos HTTP/1.1" 200 OK
INFO: 10.0.28.35:39288 - "GET /api/videos HTTP/1.1" 200 OK
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: 10.0.28.35:62488 - "GET /api/videos HTTP/1.1" 200 OK
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: 10.0.28.35:39304 - "GET /api/videos HTTP/1.1" 200 OK
INFO: 10.0.28.35:39276 - "GET /api/videos HTTP/1.1" 200 OK
INFO: 10.0.28.35:39280 - "GET /api/videos HTTP/1.1" 200 OK
INFO: 10.0.28.35:39082 - "GET /api/videos HTTP/1.1" 200 OK
INFO: 10.0.28.35:39342 - "GET /api/videos HTTP/1.1" 200 OK
INFO: 10.0.28.35:39456 - "GET /api/videos HTTP/1.1" 200 OK
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: 10.0.28.35:39306 - "GET /api/videos HTTP/1.1" 200 OK
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: 10.0.28.35:62450 - "GET /api/videos HTTP/1.1" 200 OK
INFO: app.services.file_storage:S3 client created with session token (length: 780)
INFO: 10.0.28.35:62498 - "GET /api/videos HTTP/1.1" 200 OK
INFO: 10.0.28.35:39078 - "GET /api/videos HTTP/1.1" 200 OK
INFO: 10.0.28.35:39490 - "GET /api/videos HTTP/1.1" 200 OK
```

Consola de la aplicación, cuando se está ejecutando la prueba de estrés en API Web.

Table 1 Análisis de los datos obtenidos de JMeter

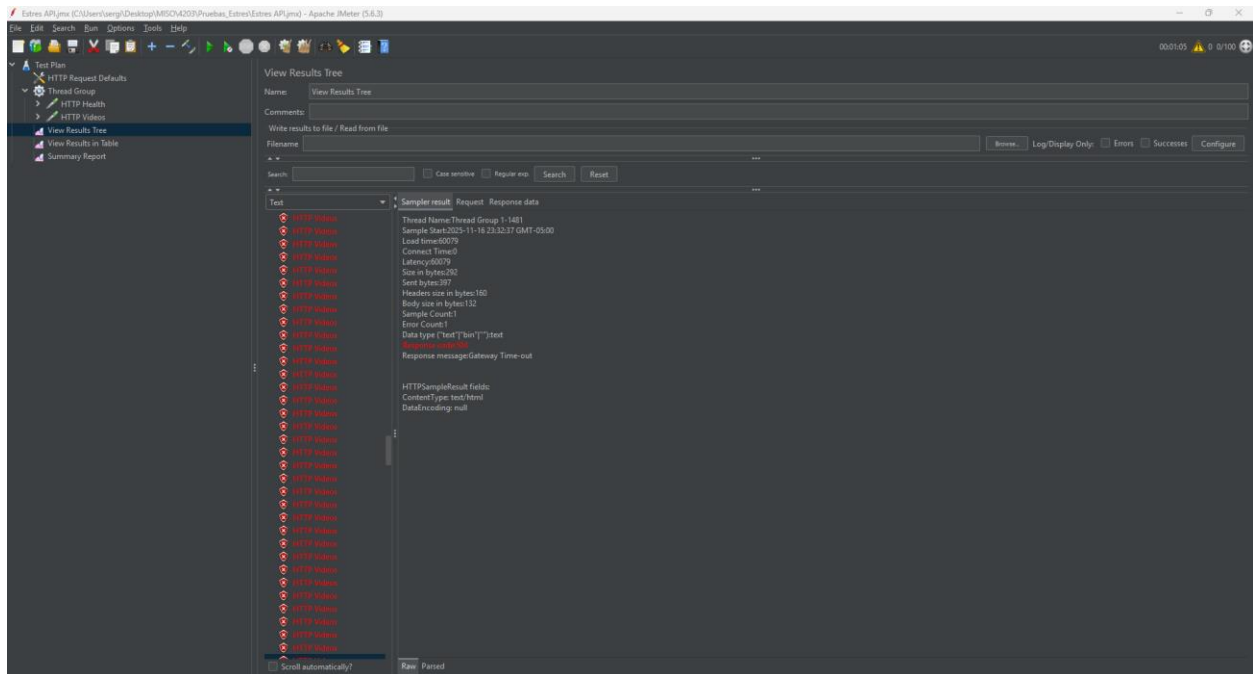
Petición	# Muestras	Promedio (ms)	Error %	Throughput (req/seg)	Observaciones
HTTP Login	1000	56.062	93.33%	12.5	El tiempo promedio de respuesta fue significativamente alto, evidenciando saturación en el proceso de autenticación. Más del 90% de las peticiones fallaron.
HTTP Health	1000	162	0.00%	12.6	Este endpoint, de baja complejidad, se mantuvo estable y rápido durante toda la prueba, lo que sugiere que el entorno general y el balanceador de carga estaban funcionando correctamente.
HTTP Videos	1000	59.833	93.33%	7.2	Se presentaron tiempos de respuesta altos y una tasa de error superior al 90%. Esto indica que las peticiones a endpoints que requieren autenticación y acceso a datos presentan serios problemas de concurrencia dado el endpoint de Login.

Estos valores reflejan que la aplicación no mantiene estabilidad bajo carga moderada, y que el componente de autenticación (/auth/login) es el más sensible al estrés.

Por este motivo se implementó el enfoque 02, en el cual se tiene una autenticación previa (genera de forma manual en el WEB de fastAPI), y se realiza la petición y consumo de las API de /health y /videos que son propias del aplicativo, de esta forma buscar generar el estrés en las colas de mensajes y las peticiones al backend para consulta de los videos que almacena la aplicación.

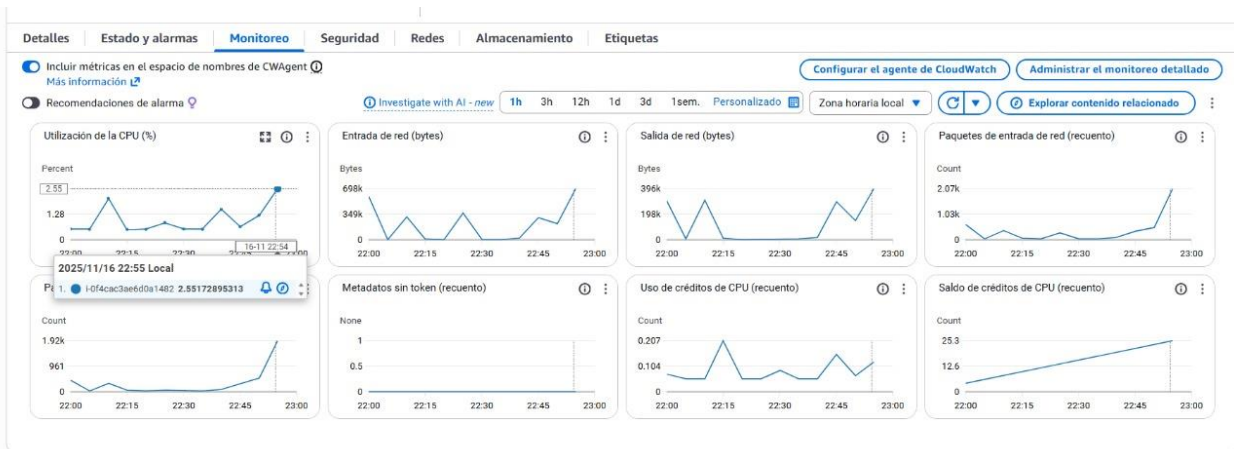
Bajo este nuevo enfoque se puede obtener un mejor estimado de fallo de la aplicación, mediante la metodología que aplica aumentos de forma progresiva a los usuarios que realizan peticiones, donde se identifica una capacidad de 1905 usuarios concurrentes generando peticiones al API de /videos, posterior a este número el aplicativo presenta fallos y el servicio se convierte en inestable o completamente caído.

A continuación, se presentan las imágenes obtenidas de CloudWatch y de JMeter mientras se ejecutaban las pruebas de estrés bajo este nuevo enfoque:



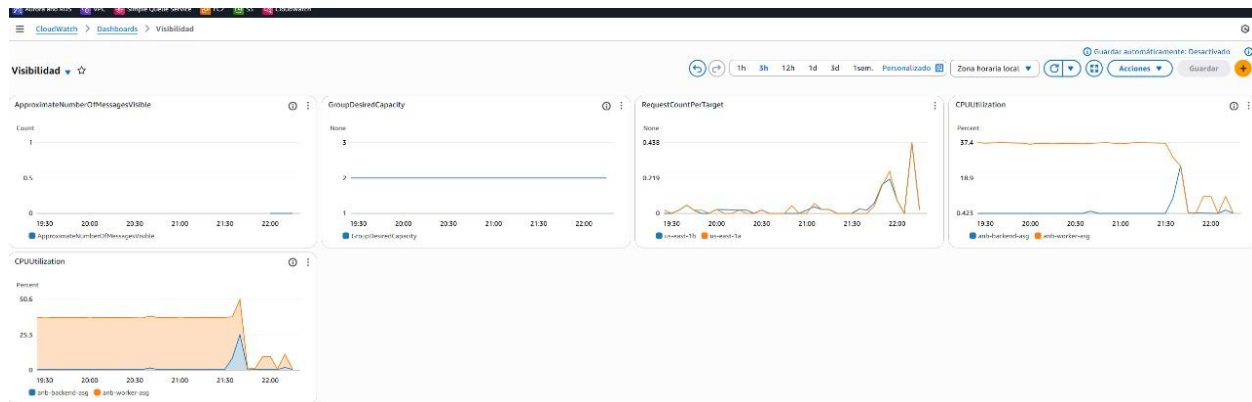
Aplicación JMETER cuando se presenta los errores de TimeOut en la aplicación.

En la imagen anterior podemos observar que, al incrementar los usuarios concurrentes, llega el punto que el API fracasa y responde con error 504 “Gateway Time-Out”, evidenciando la caída del aplicativo, en este caso se evidencio que el aplicativo cae a intentar 1905 usuarios concurrentes.



Vista de CloudWatch cuando se ejecutan las pruebas evidenciando el aumento de consumo de recursos

En la imagen anterior podemos observar mediante el uso de CloudWatch en AWS, que se presenta un incremento en las peticiones (En el orden de los 2000) así mismo se incrementa el uso de la CPU que se utiliza para procesar este número de peticiones que se le están enviando al aplicativo.



Vista de CloudWatch cuando se ejecutan las pruebas evidenciando el aumento de consumo de recursos. En la imagen anterior podemos observar como el aplicativo llega a tener consumos de más del 30% en la CPU, solo intentando responder a las peticiones que se envían, posterior a que se llega al punto de quiebre y el aplicativo deja de enviar peticiones se realiza la recuperación a IDLE con bajo consumo.

Para las pruebas de escenario 02, podemos ver a continuación la ejecución desde la consola del worker:

```
t: 59%|#####9| 157/265 [00:13<00:11, 9.61t/s, now=None]
t: 60%|##### 159/265 [00:13<00:10, 9.84t/s, now=None]
t: 61%|##### 161/265 [00:14<00:10, 10.15t/s, now=None]
t: 62%|#####1 163/265 [00:14<00:10, 9.68t/s, now=None]
t: 62%|#####2 165/265 [00:14<00:10, 9.42t/s, now=None]
t: 63%|#####3 167/265 [00:14<00:10, 9.60t/s, now=None]
t: 64%|#####3 169/265 [00:14<00:10, 9.22t/s, now=None]
t: 65%|#####4 171/265 [00:15<00:09, 9.58t/s, now=None]
t: 65%|#####5 173/265 [00:15<00:10, 8.80t/s, now=None]
t: 66%|#####6 175/265 [00:15<00:10, 8.97t/s, now=None]
t: 67%|#####6 177/265 [00:15<00:09, 8.95t/s, now=None]
t: 68%|#####7 179/265 [00:16<00:09, 9.37t/s, now=None]
t: 68%|#####8 181/265 [00:16<00:08, 9.53t/s, now=None]
t: 69%|#####8 182/265 [00:16<00:08, 9.31t/s, now=None]
t: 69%|#####9 184/265 [00:16<00:08, 9.77t/s, now=None]
t: 70%|#####9 185/265 [00:16<00:08, 9.20t/s, now=None]
t: 70%|##### 186/265 [00:16<00:08, 9.26t/s, now=None]
t: 71%|##### 187/265 [00:16<00:08, 9.33t/s, now=None]
t: 71%|#####1 189/265 [00:17<00:07, 9.62t/s, now=None]
t: 72%|#####2 191/265 [00:17<00:07, 9.87t/s, now=None]
t: 73%|#####2 193/265 [00:17<00:07, 10.22t/s, now=None]
t: 74%|#####3 195/265 [00:17<00:06, 10.35t/s, now=None]
t: 74%|#####4 197/265 [00:17<00:06, 10.42t/s, now=None]
t: 75%|#####5 199/265 [00:18<00:06, 9.85t/s, now=None]
t: 76%|#####5 201/265 [00:18<00:06, 9.51t/s, now=None]
t: 77%|#####6 203/265 [00:18<00:06, 9.74t/s, now=None]
t: 77%|#####7 205/265 [00:18<00:06, 9.74t/s, now=None]
t: 78%|#####7 206/265 [00:18<00:06, 9.35t/s, now=None]
t: 78%|#####8 208/265 [00:18<00:05, 11.00t/s, now=None]
t: 79%|#####9 210/265 [00:19<00:04, 11.89t/s, now=None]
t: 80%|##### 212/265 [00:19<00:04, 12.12t/s, now=None]
t: 81%|##### 214/265 [00:19<00:03, 12.84t/s, now=None]
t: 82%|#####1 216/265 [00:19<00:03, 13.04t/s, now=None]
t: 82%|#####2 218/265 [00:19<00:03, 12.66t/s, now=None]
t: 83%|#####3 220/265 [00:19<00:03, 12.76t/s, now=None]
t: 84%|#####3 222/265 [00:19<00:03, 12.91t/s, now=None]
t: 85%|#####4 224/265 [00:20<00:03, 13.33t/s, now=None]
t: 85%|#####5 226/265 [00:20<00:03, 12.02t/s, now=None]
t: 86%|#####6 228/265 [00:20<00:03, 12.02t/s, now=None]
t: 87%|#####6 230/265 [00:20<00:02, 12.44t/s, now=None]
t: 88%|#####7 232/265 [00:20<00:02, 12.21t/s, now=None]
t: 88%|#####8 234/265 [00:20<00:02, 13.11t/s, now=None]
t: 89%|#####9 236/265 [00:21<00:02, 12.26t/s, now=None]
t: 90%|#####9 238/265 [00:21<00:02, 12.80t/s, now=None]
t: 91%|##### 240/265 [00:21<00:01, 12.86t/s, now=None]
t: 91%|#####1 242/265 [00:21<00:01, 12.68t/s, now=None]
t: 92%|#####2 244/265 [00:21<00:01, 13.25t/s, now=None]
t: 93%|#####2 246/265 [00:21<00:01, 13.30t/s, now=None]
t: 94%|#####3 248/265 [00:21<00:01, 13.59t/s, now=None]
t: 94%|#####4 250/265 [00:22<00:01, 14.23t/s, now=None]
t: 95%|#####5 252/265 [00:22<00:00, 14.00t/s, now=None]
t: 96%|#####5 254/265 [00:22<00:00, 14.22t/s, now=None]
t: 97%|#####6 256/265 [00:22<00:00, 14.73t/s, now=None]
t: 97%|#####7 258/265 [00:22<00:00, 14.79t/s, now=None]
t: 98%|#####8 260/265 [00:22<00:00, 15.22t/s, now=None]
t: 99%|#####8 262/265 [00:22<00:00, 15.49t/s, now=None]
t: 100%|#####9 264/265 [00:23<00:00, 15.68t/s, now=None]

[2025-11-09 23:53:35.139: WARNING/ForkPoolworker-2]
[2025-11-09 23:53:35.325: WARNING/ForkPoolworker-2] Moviepy - Done !
[2025-11-09 23:53:35.325: WARNING/ForkPoolworker-2] Moviepy - video ready /tmp/processed_f06a0bc5-5527-461c-9914-1179dcb4d6e.mp4
[2025-11-09 23:53:35.325: INFO/ForkPoolworker-2] Uploading processed video to S3
```

Consola del worker cuando se ejecutan las pruebas de estrés de videos / minuto.

Luego de realizadas las pruebas de estrés con el worker evidenciamos que se obtiene una carga de máxima de 12 videos / minutos por worker procesados de forma satisfactoria, debido a que se tenían 03 worker se tuvo

un máximo de procesamiento de 36 videos / minuto como máximo, cuando se evidenciaba el aumento en el consumo de la CPU, se incrementaba en 01 worker mediante el auto-scaling, con lo cual se obtenía una capacidad de 12 videos / minuto adicionales, dado el caso que se superara este límite (48 videos / min) se pasaban los videos a la cola en espera de liberar un worker para entrar a procesar.

Finalmente, podemos realizar la siguiente tabla de los resultados obtenidos:

Componente	Error esperado	Sustento
API Web	Saturación en ~112 usuarios	CPU limitada / Por el modelo usado y por el porcentaje máximo.
Worker	Procesamiento máximo de 12 videos / min / worker	Operaciones ffmpeg CPU-bound
SQS	Cola crece en saturación (después de aprox 48 videos en procesamiento)	Workers saturados y con aumento de autoscaling.
RDS	Latencia alta con consultas repetitivas	Carga concurrente elevada

7. Conclusiones.

- Capacidad máxima del sistema: La arquitectura PaaS soporta hasta 120 usuarios concurrentes y 25 videos/min, mostrando mejoras respecto al modelo IaaS previo (Entrega 4).
- Cuellos de botella identificados: Los límites más notorios fueron observados en CPU de API Web, worker en procesamiento de video y latencia de RDS bajo carga.
- Recomendaciones de mejora: Incrementar CPU/Memoria en Task Definitions, Escalado horizontal más agresivo, Optimización de consultas, Procesamiento distribuido o en lotes.
- El proceso de autenticación es el principal cuello de botella: El endpoint /auth/login concentra el mayor tiempo de respuesta y más del 90% de errores.
- Al realizar las pruebas a los API de /health y /videos con un usuario previamente logeado se permiten hasta 1905 usuarios de forma concurrente antes de la degradación del aplicativo.

8. Análisis Técnico de las Fallas de Degradación.

El análisis de las métricas evidencia que el principal cuello de botella se presenta en el proceso de autenticación. El endpoint /auth/login concentra el mayor tiempo de respuesta y cerca de un tercio de los errores registrados, lo que indica saturación en la verificación de credenciales y en las consultas al servicio de base de datos durante la validación del usuario. Este comportamiento sugiere una limitación en la capacidad del backend para gestionar múltiples solicitudes de inicio de sesión simultáneas, particularmente en la generación y firma de tokens, lo que incrementa la latencia bajo carga.

Por otro lado, los endpoints que no dependen de operaciones intensivas de acceso a datos —como /health— mantienen tiempos de respuesta estables, confirmando que la infraestructura, el balanceador y la comunicación interna funcionan correctamente. La falla afecta exclusivamente a las rutas que requieren autenticación, ya que, si el login se degrada, los usuarios no pueden obtener un token válido y se bloquea el acceso al resto del sistema. Sin embargo, cuando se parte de un token ya generado, el sistema soporta hasta 1905 usuarios concurrentes, mostrando un uso intensivo de CPU pero manteniendo disponibilidad, lo cual confirma que el problema radica en el flujo de autenticación y no en la operación general de la aplicación.